

Skill Development Course - NodeJs Lab Manual - (LAB PROGRAMS).....	2
Aim:.....	2
1 & 2 Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.....	3
Solution :.....	3
Index.html.....	3
Login.html.....	4
registration.html.....	6
cart.html.....	9
style.css.....	11
3.Aim:.....	18
☛ Use JavaScript for doing client – side validation of the pages implemented in the experiment.....	18
Solution :.....	18
☛ Client - Side validation of Registration Page.....	18
registrationJS.html.....	18
☛ Develop a java stand alone application that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.....	22
Solution :.....	22
☛ MySQL Code:.....	22
Java Code.....	23
InsertData.java.....	23
Output :.....	24
UpdateData.java.....	24
Output :.....	25
DeleteData.java.....	25
Output :.....	26
DisplayData.java.....	26
Output :.....	27
Aim:.....	27
☛ Create a service in react that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js.....	28
Solution :.....	28
Step 1: Create an OpenWeatherMap Account and Generate API Key.....	28
Step 2: Set up a new React project.....	29
Step 3: Install required packages.....	29
Step 4: Create a Weather component.....	30

Step 5: Connect the Weather component to your app.....	31
Aim:.....	33
☛ Create a TODO application in react with necessary components and deploy it into github.....	33
Solution :.....	34
Step 1: Set Up the Project.....	34
1. Create a React App:.....	34
2. Navigate to the Project Directory:.....	34
3. Start the Development Server:.....	34
Step 2: Create the App Component.....	35
Step 3: Create the TodoList.....	35
src->Component.....	35
Step 4: Create the place the TodoItem in.....	37
src->Component.....	37
Step 5: Styling.....	38
Output :.....	38
Initially it looks like:.....	38
Next,.....	38
Aim:.....	39
☛ Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman).....	39
Solution :.....	39
db.js.....	40
app.js.....	41
Output :.....	44
GET:.....	44
POST : Create a New Student.....	44

Skill Development Course - NodeJs Lab Manual - (LAB PROGRAMS)

Aim:

1 & 2 Build a responsive [web application](#) for [shopping](#) cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.

Solution :

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <link rel="stylesheet" href="/style.css">
  <title>Home - FBS</title>
</head>
<body>
  <div class="wrapper">
    <div class="container">
      <header>
        <table width="100%" align="center" cellpadding="0" cellspacing="2">
          <tr>
            <th width="20%"></th>
            <th colspan=4>
              <h1 style="color:white;">FBS - WORLD BEST ONLINE EBOOKS WEBSITE</h1>
            </th>
          </tr>
        </table>
      </header>
      <nav>
        <table width="100%" align="center" cellpadding="0" cellspacing="2">
          <tbody align="center" style="font-weight:bold;font-size:18px;">
            <tr>
              <td width="20%"><hr><a href="index.html">Home</a><hr></td>
              <td width="20%"><hr><a href="login.html">Login</a><hr></td>
              <td width="20%"><hr><a href="registration.html">Registration</a><hr></td>
              <td width="20%"><hr><a href="cart.html" >Cart</a><hr></td>
            </tr>
          </tbody>
        </table>
      </nav>
    </div>
  </div>
```

```

<div class="container1">
  <div class="sidebar1"></div>
  <div class="container2">
    <main>
      <center>
        <h2>Welcome to FBS e-Book's Website</h2>
        <p>Shopping at <font size=5>FBS</font> can be both <font size=5>fun</font>
          and <font size=5>savings</font>.</br>Shop with us in this special <font
            size=5>discount</font> season and save upto <font size=5>90%</font> on all
your
          purchases.</br></p>
        <br/><br/><br/><br/><br/><br/><br/>
      </main>
    </div>
    <div class="sidebar2"></div>
  </div>
  <footer><font color="white">(C) 2024 All rights reserved by FBS ebooks</font></footer>
</div>
</body>
</html>

```

Login.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <link rel="stylesheet" href="./style.css">
  <title>Login - FBS</title>
</head>
<body>
  <div class="wrapper">
    <div class="container">
      <header>
        <table width="100%" align="center"
cellpadding="0" cellspacing="2">
          <tr>
            <th width="20%"></th>
            <th colspan=4>
              <h1 style="color:white;">FBS - WORLD BEST ONLINE
EBOOKS WEBSITE</h1>

```

```

        </th>
    </tr>
</table>
</header>
<nav>
    <table width="100%" align="center"
cellpadding="0" cellspacing="2">
        <tbody align="center"
style="font-weight:bold;font-size:18px;">
            <tr>
                <td width="20%"><hr><a
href="index.html">Home</a><hr></td>
                <td width="20%"><hr><a
href="login.html">Login</a><hr></td>
                <td width="20%"><hr><a
href="registration.html">Registration</a><hr></td>
                <td width="20%"><hr><a href="cart.html"
>Cart</a><hr></td>
            </tr>
        </tbody>
    </table>
</nav>
</div>
<div class="container1">
    <div class="sidebar1"></div>
    <div class="container2">
        <main>
            <center><br>
            <h3> Login Details</h3> <br/>
            <form name="f1">
                <table width="100%" align="center" >
                    <tr>
                        <td> User Name : </td>
                        <td> <input type="text"
name="username"></td>
                    </tr>
                    <tr><td><br></td></tr>
                    <tr>
                        <td> Password : </td>
                        <td> <input type="password"
name="password"></td>

```

```

        </tr>
        <tr><td><br></td></tr>
        <tr><td></td>
        <td><input type="submit"
value="SUBMIT">
        <input type="reset"
value="RESET"></td>
        </tr>
    </table>
</form>
</center>
</main>
</div>
<div class="sidebar2"></div>
</div>
<footer><font color="white">(C) 2024 All rights reserved
by FBS ebooks</font></footer>
</div>
</body>
</html>

```

registration.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="stylesheet" href="./style.css">
    <title>Registration - FBS</title>
</head>
<body>
    <div class="wrapper">
        <div class="container">
            <header>
                <table width="100%" align="center"
cellpadding="0" cellspacing="2">
                    <tr>
                        <th width="20%"></th>

```

```

        <th colspan=4>
        <h1 style="color:white;">FBS - WORLD BEST ONLINE
EBOOKS WEBSITE</h1>
        </th>
    </tr>
</table>
</header>
<nav>
    <table width="100%" align="center"
cellpadding="0" cellspacing="2">
        <tbody align="center"
style="font-weight:bold;font-size:18px;">
            <tr>
                <td width="20%"><hr><a
href="index.html">Home</a><hr></td>
                <td width="20%"><hr><a
href="login.html">Login</a><hr></td>
                <td width="20%"><hr><a
href="registration.html">Registration</a><hr></td>
                <td width="20%"><hr><a href="cart.html"
>Cart</a><hr></td>
            </tr>
        </tbody>
    </table>
</nav>
</div>
<div class="container1">
    <div class="sidebar1"></div>
    <div class="container2">
        <main>
            <center><br>
                <h3>Registration Form </h3>
                <br/>
                <form name="f1">
                    <table cellpadding="1" align="center" >
                        <tr><td> Name:*</td>
                        <td><input type="text"
name="username"></td></tr>
                        <tr><td>Password:*</td>
                        <td><input type="password"
name="password"></td></tr>

```

[illegible]


```

        <footer><font color="white">(C) 2024 All rights reserved
by FBS ebooks</font></footer>
    </div>
</body>
</html>

```

cart.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="stylesheet" href="./style.css">
    <title>Cart - FBS</title>
</head>
<body>
    <div class="wrapper">
        <div class="container">
            <header>
                <table width="100%" align="center"
cellpadding="0" cellspacing="2">
                    <tr>
                        <th width="20%"></th>
                        <th colspan=4>
                            <h1 style="color:white;">FBS - WORLD BEST ONLINE
EBOOKS WEBSITE</h1>
                        </th>
                    </tr>
                </table>
            </header>
            <nav>
                <table width="100%" align="center"
cellpadding="0" cellspacing="2">
                    <tbody align="center"
style="font-weight:bold;font-size:18px;">
                        <tr>
                            <td width="20%"><hr><a
href="index.html">Home</a><hr></td>

```

```

        <td width="20%"><hr><a
href="login.html">Login</a><hr></td>
        <td width="20%"><hr><a
href="registration.html">Registration</a><hr></td>
        <td width="20%"><hr><a href="cart.html"
>Cart</a><hr></td>
    </tr>
</tbody>
</table>
</nav>
</div>
<div class="container1">
    <div class="sidebar1"></div>
    <div class="container2">
        <main>
            <center>
                <h3>Cart</h3>
                <table width="100%" align="center" >
                    <tbody>
                        <tr>
                            <th width="40%"><hr>BookName<hr></th>
                            <th width="20%"><hr>Price<hr></th>
                            <th width="20%"><hr>Quantity<hr></th>
                            <th width="20%"><hr>Amount<hr></th>
                        </tr>
                        </tbody>
                        <tbody align=center>
                            <tr> <td>Java Programming </td>
                                <td>Rs. 2300/-</td>
                                <td>2</td>
                                <td>Rs. 4600/-</td></tr>
                            <tr><td>Web Technologies</td>
                                <td>Rs. 3000/-</td>
                                <td>1</td>
                                <td>Rs. 3000/-</td></tr>
                            <tr><td></td>
                                <td><hr><font color="#996600">Total
Amount:</font><hr></td>
                                <td><hr>3<hr></td>
                                <td><hr>Rs. 7600/-<hr></td> </tr>
                        </tbody>

```

```
                </table>
            </center>
        </main>
    </div>
    <div class="sidebar2"></div>
</div>
    <footer><font color="white">(C) 2024 All rights reserved
by FBS ebooks</font></footer>
</div>
</body>
</html>
```

style.css

```
body{
    font-family: monospace;
}

main {
    background-color: #efefef;
    color: #330000;
    margin-left: 10px;
    height: 60vh;
}

header, footer {
    background-color: #000d57;
    color: #fff;
    padding: 1rem;
    height: 50px;
}

header, nav{
```

```
margin-bottom: 10px;
flex-basis: 50%;
}
```

```
footer{
margin-top: 10px;
}
```

```
nav {
background-color: #fff;
color: #000;
padding: 1rem;
height: 20px;
}
```

```
.sidebar1, .sidebar2 {
flex-basis: 10%;
background-color: #fff;
color: #000;
}
```

```
.sidebar2{
margin-left: 10px;
}
```

```
.container1{
display: flex;
}
```

```
.container2 {
display: flex;
flex-direction: column;
flex: 1;
```

```
}
```

```
header, nav, main, .sidebar1, .sidebar2, footer{
```

```
  display: flex;
```

```
  align-items: center;
```

```
  justify-content: center;
```

```
  border-radius: 10px;
```

```
}
```

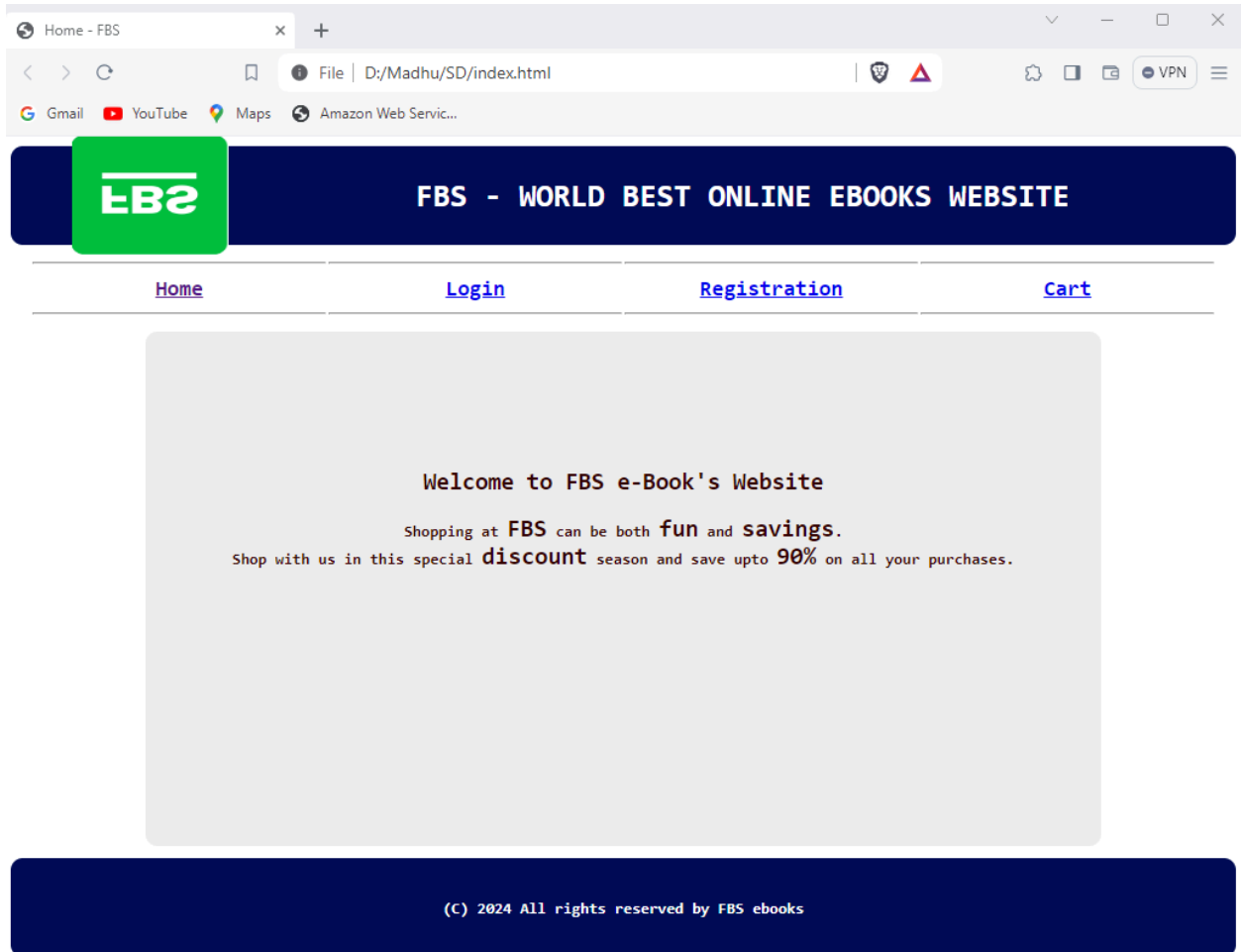
```
.wrapper {
```

```
  display: flex;
```

```
  flex-direction: column;
```

```
  font-weight: 600;
```

```
}
```



Login - FBS

File | D:/Madhu/SD/login.html

Gmail YouTube Maps Amazon Web Servic...



FBS - WORLD BEST ONLINE EBOOKS WEBSITE

[Home](#)

[Login](#)

[Registration](#)

[Cart](#)

Login Details

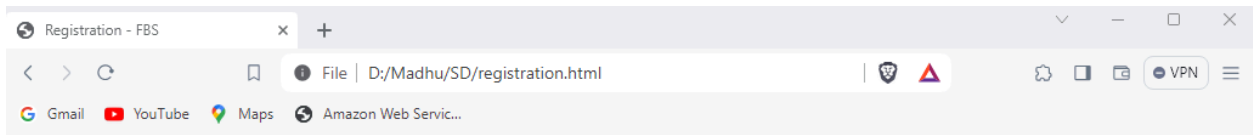
User Name :

Password :

SUBMIT

RESET

(C) 2024 All rights reserved by FBS ebooks



FBS - WORLD BEST ONLINE EBOOKS WEBSITE

[Home](#)

[Login](#)

[Registration](#)

[Cart](#)

Registration Form

Name:*

Password:*

Email ID:*

Phone Number:*

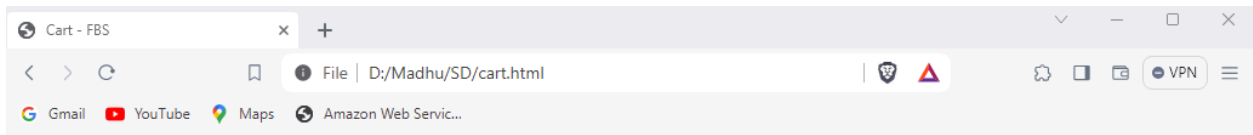
Gender:* ☐ Male ☐ Female

Language Known:* ☐ English
☐ Telugu
☐ Hindi
☐ Tamil

Address:*

*fields are mandatory

(C) 2024 All rights reserved by FBS ebooks



FBS - WORLD BEST ONLINE EBOOKS WEBSITE

[Home](#)

[Login](#)

[Registration](#)

[Cart](#)

Cart

BookName	Price	Quantity	Amount
Java Programming	Rs. 2300/-	2	Rs. 4600/-
Web Technologies	Rs. 3000/-	1	Rs. 3000/-
Total Amount:		3	Rs. 7600/-

(C) 2024 All rights reserved by FBS ebooks

3.Aim:

👉 Use JavaScript for doing client – side validation of the pages implemented in the experiment

Solution :

👉 Client - Side validation of Registration Page

registrationJS.html

```
<html>
<head>
    <title> Welcome to NNRG e-Book's website</title>
    <script language="javascript">
        function validate() {
            // username validation
            var uname = f1.username.value;
            if (uname.length<=0)
            {
                alert("Please Enter UserName");
                f1.username.focus();
                return false;
            }
            if (uname.length < 8)
            {
                alert("Please enter UserName not less than 8");
                f1.username.focus();
                return false;
            }
            //password validation
            var pwd = f1.password.value;
            if (pwd.length<=0)
            {
                alert("Please Enter password");
                f1.password.focus();
                return false;
            }
        }
    </script>
</head>
<body>
    <div>
        <div>
            <input type="text" value="Username" />
            <input type="password" value="Password" />
            <input type="button" value="Register" />
        </div>
    </div>
</body>
</html>
```

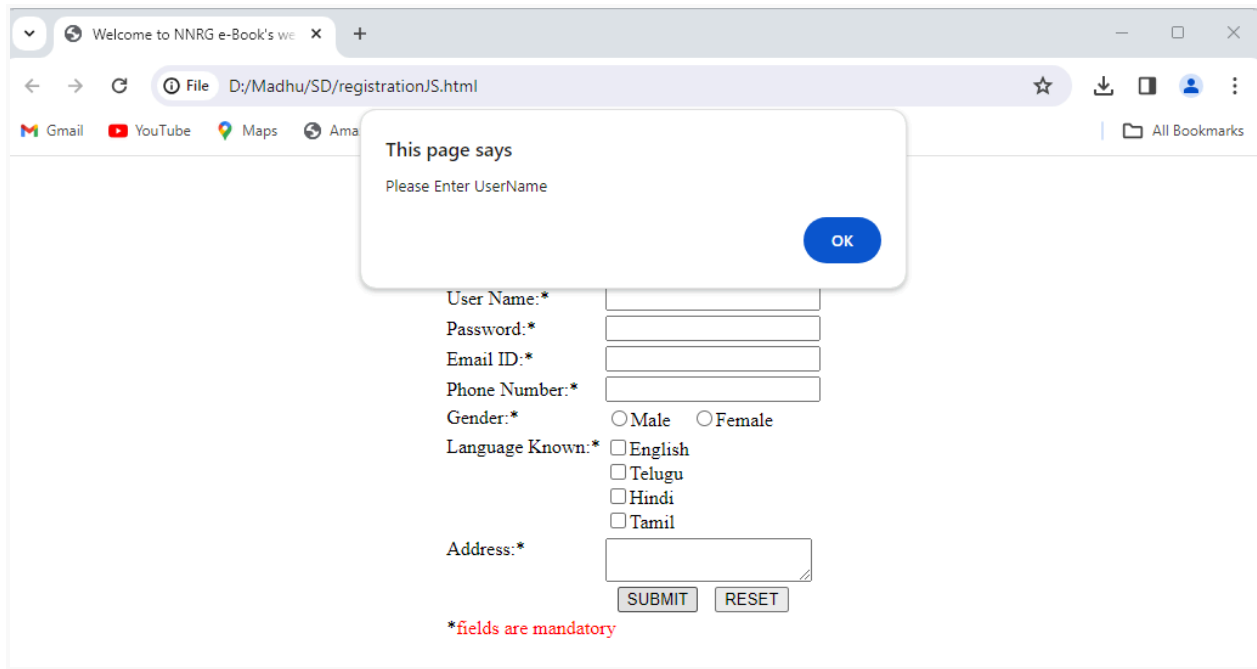
```
if (pwd.length < 6)
{
    alert("Please enter Password not less than 6");
    f1.password.focus();
    return false;
}
// email validation
var email = f1.email.value;
if (email.length<=0)
{
    alert("Please Enter email");
    f1.email.focus();
    return false;
}
else {
    let eflag=false;
    for (i=0;i<email.length;i++) {
        if(email.charAt(i)=="@")
        {
            eflag=true;
        }
    }
    if(!(eflag))
    {
        alert("Please enter a valid Email ID");
        f1.email.focus();
        return false;
    }
}
// phone number validation
var phno = f1.phno.value;
if (phno.length<=0)
{
    alert("Please Enter Phone Number");
    f1.phno.focus();
    return false;
}
if (isNaN(phno))
{
    alert("Please Enter Valid Phone Number");
    f1.phno.focus();
}
```

```
        return false;
    }
    if (phno.length != 10)
    {
        alert("Please Enter Valid Phone Number");
        f1.phno.focus();
        return false;
    }
    // gender validation
    let flag=false;
    for(i=0;i<f1.gen.length;i++)
        if(f1.gen[i].checked)
            flag=true;
    if(!(flag))
    {
        alert("Please choose a Gender");
        return false;
    }
    // Language validation
    flag=false;
    for(i=0;i<f1.lang.length;i++)
        if(f1.lang[i].checked)
            flag=true;
    if(!(flag))
    {
        alert("Please select at least one of the Language
options.");
        return false;
    }
    // address validation
    var addr = f1.address.value;
    if (addr.length<=0)
    {
        alert("Please Enter address");
        f1.address.focus();
        return false;
    }
    // to display Success message
    alert("Registration Successful");
}
</script>
```

```

</head>
<body>
<center><br>
    <h3>Registration Form </h3>
    <br/>
    <form name="f1">
        <table cellpadding="1" align="center" >
            <tr><td> User Name:*</td>
            <td><input type="text" name="username"></td></tr>
            <tr><td>Password:*</td>
            <td><input type="password" name="password"></td></tr>
            <tr><td>Email ID:*</td>
            <td><input type="text" name="email"></td></tr>
            <tr><td>Phone Number:*</td>
            <td><input type="text" name="phno"></td></tr>
            <tr><td valign="top">Gender:*</td>
            <td><input type="radio" name="gen" value="Male">Male
            &nbsp;   <input type="radio" name="gen"
            value="Female">Female</td></tr>
            <tr> <td valign="top">Language Known:*</td>
            <td> <input type="checkbox" name="lang"
            value="English">English<br/>
            <input type="checkbox" name="lang" value="Telugu">Telugu<br>
            <input type="checkbox" name="lang" value="Hindi">Hindi<br>
            <input type="checkbox" name="lang" value="Tamil">Tamil
            </td></tr>
            <tr> <td valign="top">Address:*</td>
            <td><textarea name="address"></textarea></td>
            <tr><td></td><td><input type="button" value="SUBMIT"
            hspace="10" onclick="validate()">
            <input type="reset" value="RESET"></td></tr>
            <tr> <td colspan=2 >*<font color="#FF0000">fields are
            mandatory</font>
            </td>
            </tr>
        </table>
    </form>
</center>
</body>
</html>

```



4.Aim:

👉 Develop a java stand alone [application](#) that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.

Solution :

I will provide you with the MySQL code for creating the student table and make some changes to your Java code to improve it:

👉 MySQL Code:

```
sql> CREATE TABLE IF NOT EXISTS student (
```

```
s_id INT PRIMARY KEY,  
s_name VARCHAR(255),  
s_address VARCHAR(255)  
);
```

This SQL code creates a table with three columns: s_id for student ID (primary key), s_name for student name, and s_address for student address.

Java Code

InsertData.java

```
import java.sql.*;  
import java.util.Scanner;  
  
public class InsertData {  
    public static void main(String[] args) {  
        try (Connection con =  
DriverManager.getConnection("jdbc:mysql://localhost/mydb",  
"root", ""));  
            Statements = con.createStatement()) {  
  
            Scanner sc = new Scanner(System.in);  
            System.out.println("Inserting Data into student  
table:");  
  
            System.out.println("_____");  
  
            System.out.print("Enter student id: ");  
            int sid = sc.nextInt();  
            System.out.print("Enter student name: ");  
            String sname = sc.next();  
            System.out.print("Enter student address: ");  
            String saddr = sc.next();  
  
            String insertQuery = "INSERT INTO student VALUES (" +  
sid + ", '" + sname + "', '" + saddr + "')";
```

```

        s.executeUpdate(insertQuery);

        System.out.println("Data inserted successfully into
student table");

    } catch (SQLException err) {
        System.out.println("ERROR: " + err);
    }
}
}
}

```

Output :

Inserting Data into student table:

Enter student id: 101

Enter student name: John Doe

Enter student address: 123 Main Street

Data inserted successfully into student table

UpdateData.java

```

import java.sql.*;
import java.util.Scanner;

public class UpdateData {
    public static void main(String[] args) {
        try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost/mydb",
"root", ""));
            Statements = con.createStatement()) {

            Scanner sc = new Scanner(System.in);
            System.out.println("Update Data in student table:");

            System.out.println("_____");

            System.out.print("Enter student id: ");
            int sid = sc.nextInt();
            System.out.print("Enter student name: ");

```



```

        String sname = sc.next();
        System.out.print("Enter student address: ");
        String saddr = sc.next();

        String updateQuery = "UPDATE student SET s_name='" +
sname + "', s_address='" + saddr + "' WHERE s_id=" + sid;
        s.executeUpdate(updateQuery);

        System.out.println("Data updated successfully");

    } catch (SQLException err) {
        System.out.println("ERROR: " + err);
    }
}
}

```

Output :

Update Data in student table:

```

Enter student id: 101
Enter student name: Jane Doe
Enter student address: 456 Broad Street
Data updated successfully

```

DeleteData.java

```

import java.sql.*;
import java.util.Scanner;

public class DeleteData {
    public static void main(String[] args) {
        try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost/mydb",
"root", ""));
        Statements = con.createStatement()) {

```

```

        Scanner sc = new Scanner(System.in);
        System.out.println("Delete Data from student
table:");

System.out.println("_____");

        System.out.print("Enter student id: ");
        int sid = sc.nextInt();

        String deleteQuery = "DELETE FROM student WHERE
s_id=" + sid;
        s.executeUpdate(deleteQuery);

        System.out.println("Data deleted successfully");

    } catch (SQLException err) {
        System.out.println("ERROR: " + err);
    }
}
}

```

Output :

Delete Data from student table:

Enter student id: 101

Data deleted successfully

DisplayData.java

```

import java.sql.*;

public class DisplayData {
    public static void main(String[] args) {
        try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost/mydb",
"root", ""));

```

```

        Statement s = con.createStatement();

        ResultSet rs = s.executeQuery("SELECT * FROM
student");
        if (rs != null) {
            System.out.println("SID \t STU_NAME \t
ADDRESS");

System.out.println("_____");

            while (rs.next()) {
                System.out.println(rs.getString("s_id") + "
\t " + rs.getString("s_name") + " \t " +
rs.getString("s_address"));

System.out.println("_____");
            }
        }

    } catch (SQLException err) {
        System.out.println("ERROR: " + err);
    }
}
}
}

```

Output :

SID STU_NAME ADDRESS

102 Alice Smith 789 Oak Avenue

103 Bob Johnson 567 Pine Road

12 .Develop a React service to fetch weather data from openweathermap.org and display current and historical weather information using Chart.js.

Aim:

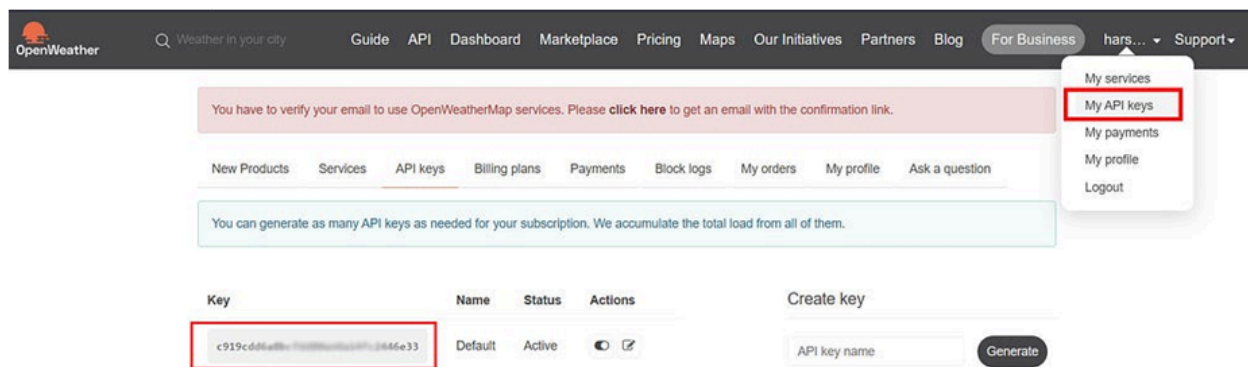
👉 Create a service in react that fetches the weather information from openweathermap.org and then display the current and historical weather information using graphical representation using chart.js

Solution :

Step 1: Create an OpenWeatherMap Account and Generate API Key

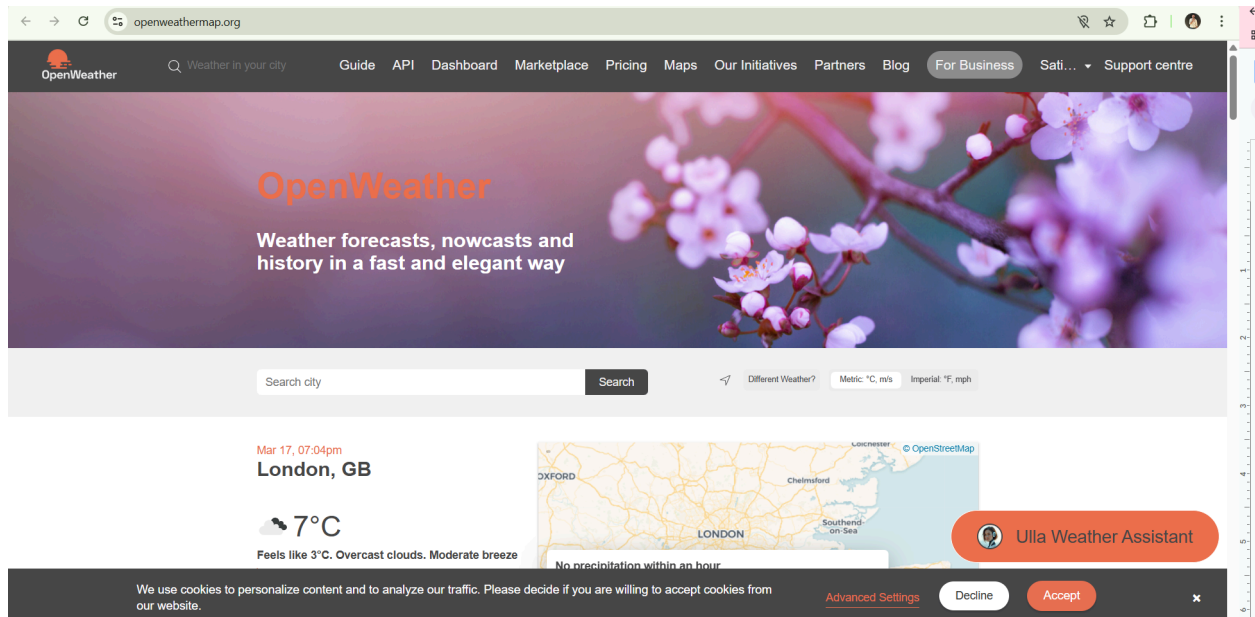
- Visit the OpenWeatherMap website (<https://openweathermap.org/>) and click on **"Sign Up"** or **"Log In"** to create an account or log into your existing account.
- Once logged in, navigate to your account dashboard.
- From the dashboard, locate my API Keys section and click on **"Create Key"** or **"API Keys"** to generate a new API key.
- Provide a name for your API key (e.g., **"WeatherApp"**) and click on the **"Generate"** or **"Create"** button.
- Your API key will be generated and displayed on the screen. Make sure to copy it as we will need it later.

Locate API key



The screenshot shows the OpenWeatherMap dashboard. At the top, there's a navigation bar with links like Guide, API, Dashboard, Marketplace, Pricing, Maps, Our Initiatives, Partners, Blog, For Business, and a user menu. The user menu is open, showing options like My services, My API keys (highlighted with a red box), My payments, My profile, and Logout. Below the navigation bar, there's a message about email verification. Underneath, there's a section for API keys with tabs for New Products, Services, API keys (selected), Billing plans, Payments, Block logs, My orders, My profile, and Ask a question. A message states: "You can generate as many API keys as needed for your subscription. We accumulate the total load from all of them." Below this, there's a table with columns: Key, Name, Status, and Actions. The first row shows a key: c919cd8fa8b170d88a5a307f26446e33, with Name: Default, Status: Active, and Actions: a toggle switch and an edit icon. To the right of the table is a 'Create key' section with a text input for 'API key name' and a 'Generate' button.

Key	Name	Status	Actions
c919cd8fa8b170d88a5a307f26446e33	Default	Active	☐



Step 2: Set up a new React project

- Open your terminal or command prompt.
- Run the following command to create a new React project:

```
npxcreate-react-app weather-app
```

- Once the project is created, navigate into the project directory:

```
cd weather-app
```

Step 3: Install required packages

In the project directory, install the necessary packages by executing the following command

```
npm install axios
```

We will use the Axios library to make HTTP requests to the OpenWeatherMap API.

Step 4: Create a Weather component

- Inside the "src" directory, create a new file called "Weather.js" and open it in your code editor.
- Add the following code to define a functional component named Weather:

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const Weather = () => {
  const [city, setCity] = useState('');
  const [weatherData, setWeatherData] = useState(null);

  const fetchData = async () => {
    try {
      const apiKey = 'c97c0c1086d42990e89d64d76f50bb61'; //
      Replace with your OpenWeatherMap API key
      const response = await axios.get(
        'https://api.openweathermap.org/data/2.5/weather?q=${city}&units=metric&appid=${apiKey}'
      );
      setWeatherData(response.data);
      console.log(response.data); //You can see all the weather
      data in console log
    } catch (error) {
      console.error(error);
    }
  };

  useEffect(() => {
    fetchData();
  }, []);

  const handleInputChange = (e) => {
    setCity(e.target.value);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
  };
}
```

```

fetchData();
};

return (
<div>
<form onSubmit={handleSubmit}>
<input
  type="text"
  placeholder="Enter city name"
  value={city}
  onChange={handleInputChange}
/>
<button type="submit">Get Weather</button>
</form>
{weatherData ? (
<>
<h2>{weatherData.name}</h2>
<p>Temperature: {weatherData.main.temp} °C</p>
<p>Description: {weatherData.weather[0].description}</p>
<p>Feels like : {weatherData.main.feels_like} °C</p>
<p>Humidity : {weatherData.main.humidity}%</p>
<p>Pressure : {weatherData.main.pressure}</p>
<p>Wind Speed : {weatherData.wind.speed}m/s</p>
</>
) : (
<p>Loading weather data...</p>
)}
</div>
);
};

```

```
export default Weather;
```

Replace {YOUR_API_KEY} in the API URL with the API key you generated from OpenWeatherMap.

Step 5: Connect the Weather component to your [app](#).

- Open the "App.js" file in the "src" directory.
- Replace the existing code with the following code:

```
import React from 'react';
import Weather from './Weather';

const App = () => {
  return (
    <div>
      <h1>Weather Forecast App</h1>
      <Weather />
    </div>
  );
};

export default App;
```

Your output should look like this:

Loading weather data...

london

Get Weather

London

Temperature: 21.83°C

Description: overcast clouds

Feels like : 21.35°C

Humidity : 49%

Pressure : 1015

Wind Speed : 2.57m/s

14. Create a TODO application in react with necessary components and deploy it into github

Aim:

👉 Create a TODO [application](#) in react with necessary components and deploy it into github.

Solution :

Step 1: Set Up the Project

Our first task is to set up the React project. This step involves creating the necessary project structure. Here's how you can do it:

1. Create a React App:

Open your terminal and navigate to your preferred directory. Run the following command to generate a new React [app](#). Replace **"todo-app"** with your desired project name:

```
npx create-react-app todo-app
```

This command will create a directory named "todo-app" with all the initial code required for a React app.

2. Navigate to the Project Directory:

Change your working directory to the "todo-app" folder:

```
cd todo-app
```

3. Start the Development Server:

Launch the development server with the following command:

```
npm start
```

This will open your React app, and you will see the default React starter page in your web browser at *'http://localhost:3000'*.

Step 2: Create the App Component

In this step, we create the App component, which serves as the entry point to our Todo List application.

```
import React from 'react';
import TodoList from '../components/TodoList';
function App() {
  return (
    <div className="App">
      <TodoList />
    </div>
  );
}
export default App;
```

Step 3: Create the TodoList

src->Component

Now, let's create the 'TodoList' component, which is responsible for managing the list of tasks and handling task-related functionality.

```
import React, { useState } from 'react';
import TodoItem from '../TodoItem';

function TodoList() {
  const [tasks, setTasks] = useState([
    {
      id: 1,
      text: 'Doctor Appointment',
      completed: true
    },
    {
      id: 2,
      text: 'Meeting at School',
      completed: false
    }
  ]);
}
```

```

    }
  ]));

const [text, setText] = useState('');
function addTask(text) {
  const newTask = {
    id: Date.now(),
    text,
    completed: false
  };
  setTasks([tasks, newTask]);
  setText('');
}
function deleteTask(id) {
  setTasks(tasks.filter(task => task.id !== id));
}
function toggleCompleted(id) {
  setTasks(tasks.map(task => {
    if (task.id === id) {
      return {task, completed: !task.completed};
    } else {
      return task;
    }
  }));
}
return (
  <div className="todo-list">
    {tasks.map(task => (
      <TodoItem
        key={task.id}
        task={task}
        deleteTask={deleteTask}
        toggleCompleted={toggleCompleted}
      />
    ))}
    <input
      value={text}
      onChange={e => setText(e.target.value)}
    />
    <button onClick={() => addTask(text)}>Add</button>
  </div>

```

```

    );
  }
export default TodoList;

```

Step 4: Create the place the TodoItem in

src->Component

In this step, we create the 'TodoItem' component, which represents an individual task in our Todo List.

```

import React from 'react';
function TodoItem({ task, deleteTask, toggleCompleted }) {
  function handleChange() {
    toggleCompleted(task.id);
  }

  return (
    <div className="todo-item">
      <input
        type="checkbox"
        checked={task.completed}
        onChange={handleChange}
      />
      <p>{task.text}</p>
      <button onClick={() => deleteTask(task.id)}>
        X
      </button>
    </div>
  );
}
export default TodoItem;

```

These three components, 'App', 'TodoList', and 'TodoItem', work together to create a functional Todo List application in React. The 'TodoList' component manages the state

of the tasks, and the 'TodoItem' component represents and handles individual tasks within the list.

Step 5: Styling

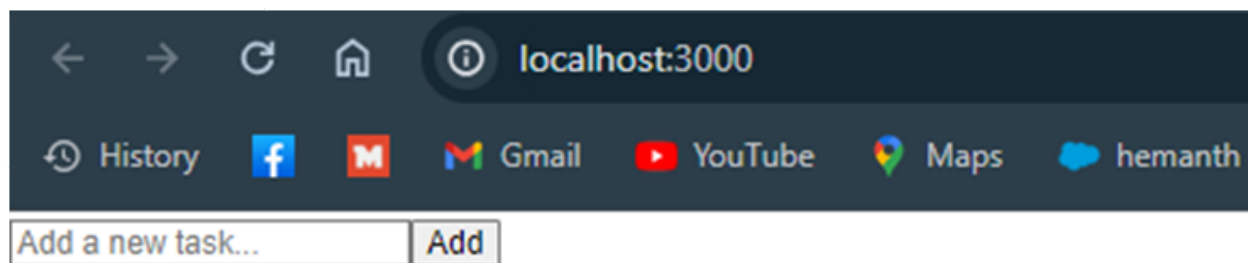
To enhance the visual appeal of your Todo List, you can apply some basic styling. Here's an example of how you can style the todo items. In the `App.css` file, add the following styles:

```
.todo-item {  
  display: flex;  
  justify-content: space-between;  
  margin-bottom: 8px;  
}  
.todo-itemp {  
  color: #888;  
  text-decoration: line-through;  
}
```

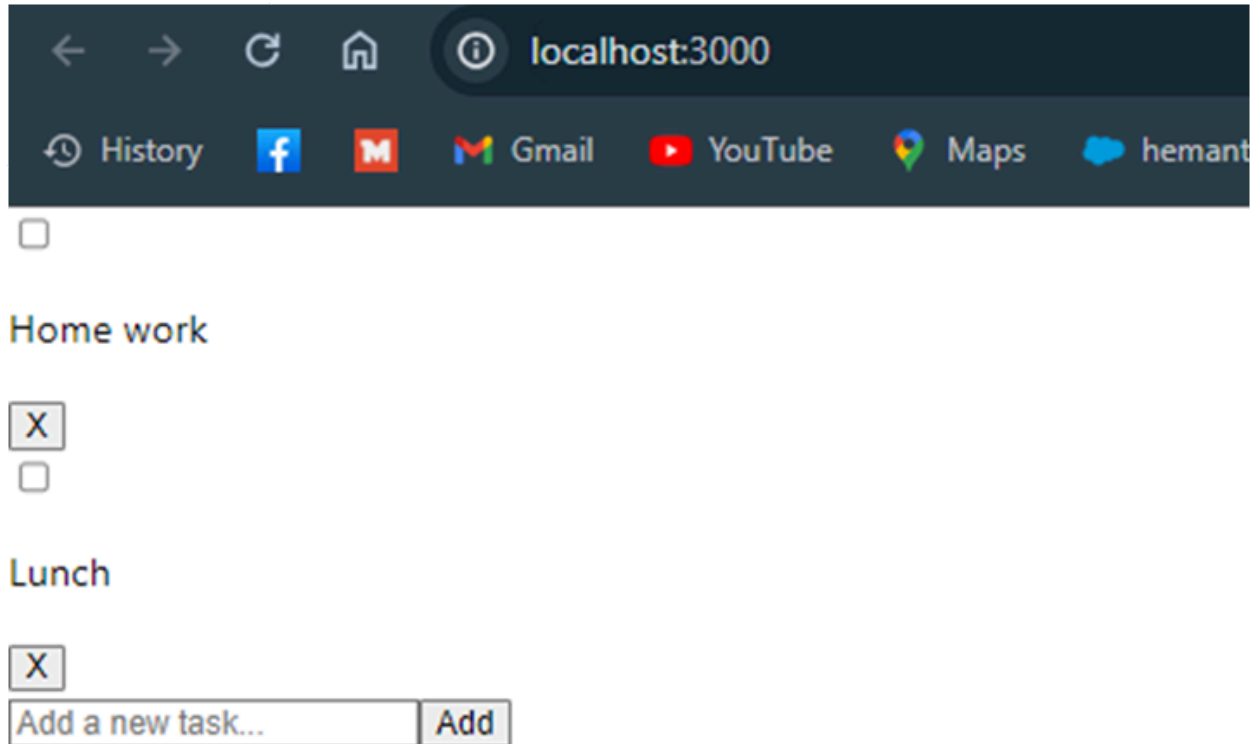
Your output should look like this:

Output :

Initially it looks like:



Next,



9. Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman).

Aim:

👉 Develop an express [web application](#) that can interact with [REST API](#) to perform CRUD operations on student data. (Use Postman)

Solution :

Firstly we need to create a new folder and open the folder in the command prompt and enter a command as below:

`npm init -y`

-
- Open that folder in the vscode by entering *code*.
- Next in the terminal we need to install all the packages we need, so we mainly use express and sqlite3.

The Command to install express and sqlite3 is
`npm install express sqlite3`

-

Then create file named as the **app.js** and **db.js**

db.js

```
const sqlite3 = require('sqlite3').verbose();

// Function to initialize the database schema
function initializeDatabase() {
  const db = new sqlite3.Database('./mydatabase.db', (err) => {
    {
      if (err) {
        console.error(err.message);
      } else {
        console.log('Connected to the SQLite database.');
```



```

        console.log('Disconnected from the SQLite
database.');
```

```

    }
  });
});
}

// Function to create the 'students' table if it doesn't exist
function createStudentsTable(db) {
  const createTableQuery = `
    CREATE TABLE IF NOT EXISTS students (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT,
      age INTEGER,
      grade TEXT
    );
  `;

  db.run(createTableQuery, (err) => {
    if (err) {
      console.error(err.message);
    } else {
      console.log('The students table has been created or
already exists.');
```

```

    }
  });
}

module.exports = { initializeDatabase };

```

when we execute both the **db.js** then the database will be created that is **mydatabase.db**

app.js

```

const express = require('express');
const sqlite3 = require('sqlite3');
const { initializeDatabase } = require('./db');
const app = express();

```

```
const port = 3000;

// Connect to SQLite database
const db = new sqlite3.Database('./mydatabase.db', (err) => {
  if (err) {
    console.log(err.message);
  } else {
    console.log('Connected to the SQLite database.');
```

```

app.post('/students', (req, res) => {
  const { name, age, grade } = req.body;
  db.run('INSERT INTO students (name, age, grade) VALUES (?, ?, ?)', [name, age, grade], function (err) {
    if (err) {
      return console.error(err.message);
    }
    res.status(201).json({ id: this.lastID });
  });
});

// Update a student
app.put('/students/:id', (req, res) => {
  const id = req.params.id;
  const { name, age, grade } = req.body;
  db.run('UPDATE students SET name = ?, age = ?, grade = ? WHERE id = ?', [name, age, grade, id], function (err) {
    if (err) {
      return console.error(err.message);
    }
    res.json({ updatedID: id });
  });
});

// Delete a student
app.delete('/students/:id', (req, res) => {
  const id = req.params.id;
  db.run('DELETE FROM students WHERE id = ?', id, function (err) {
    if (err) {
      return console.error(err.message);
    }
    res.json({ deletedID: id });
  });
});

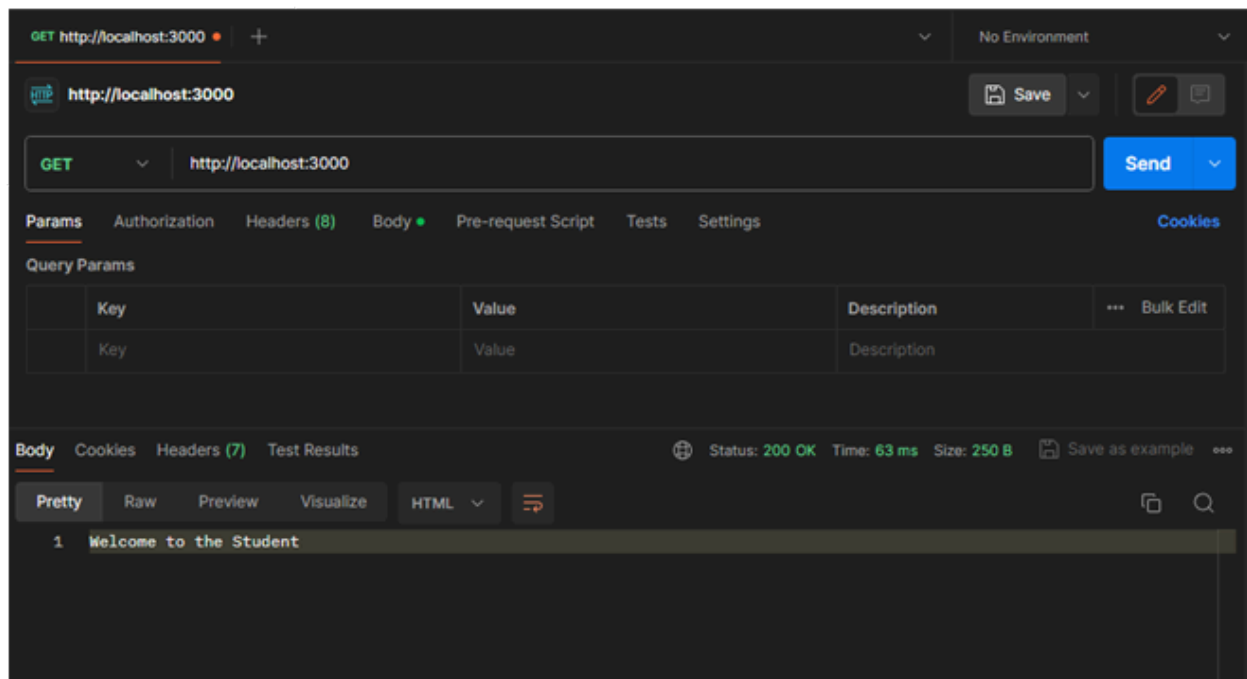
app.listen(port, () => {
  console.log('Server running at http://localhost:${port}');
});

```

Output :

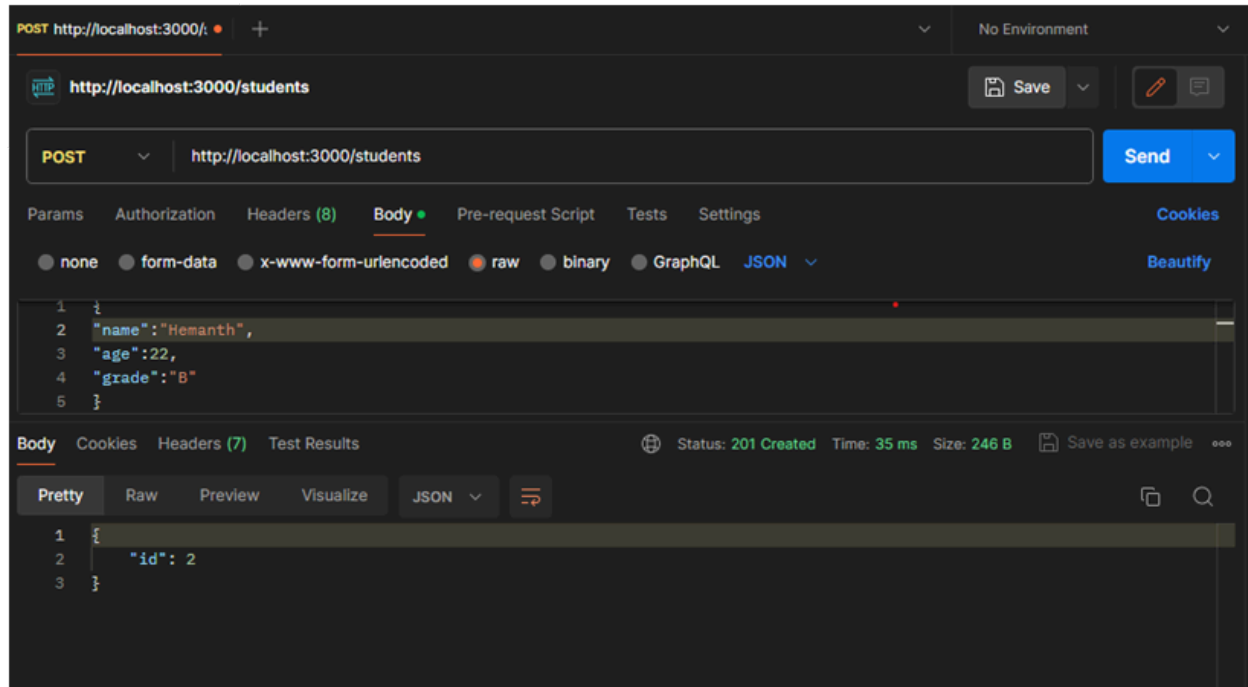
GET:

- Open Postman.
- Set the request type to GET.
- Enter the URL: *http://localhost:3000/students*



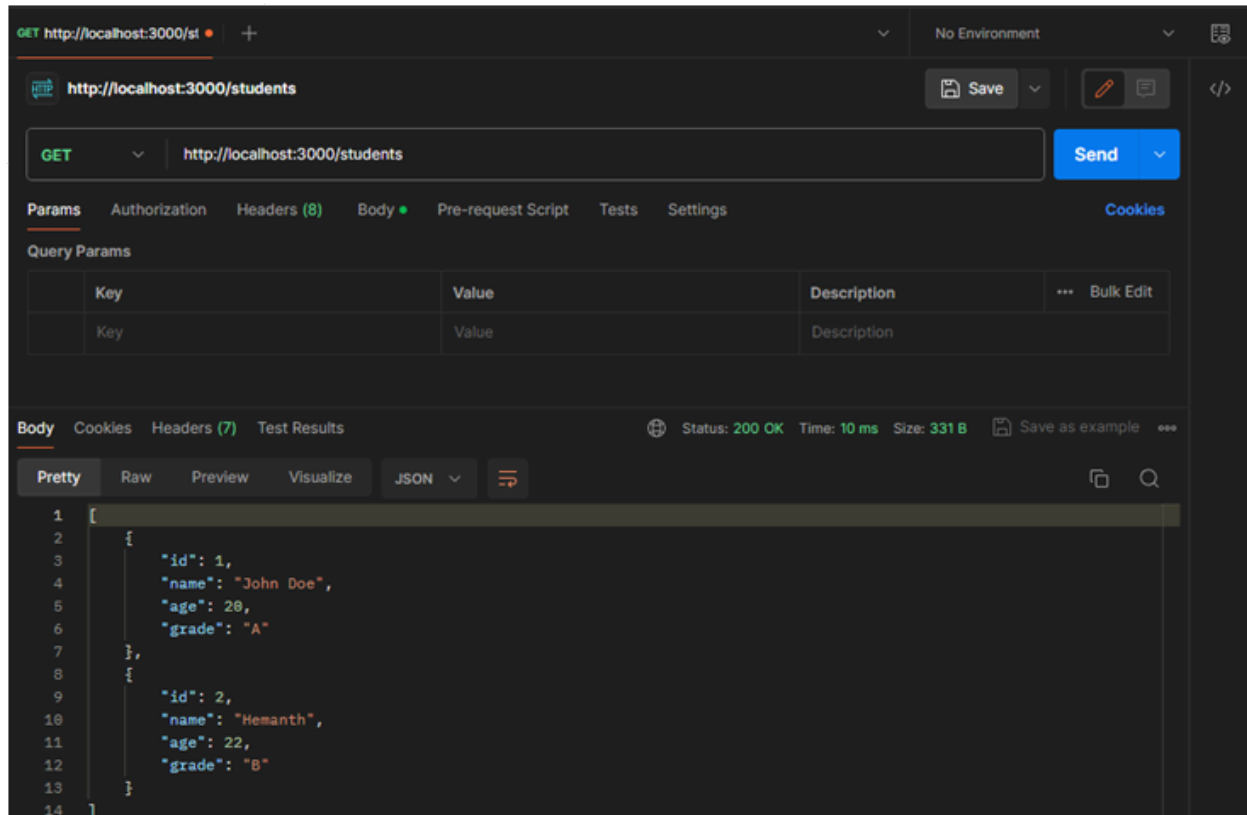
POST : Create a New Student

- Open Postman.
- Set the request type to POST.
- Enter the URL: *http://localhost:3000/students*.
- Go to the **"Body"** tab.
- Select raw and set the body to JSON format.



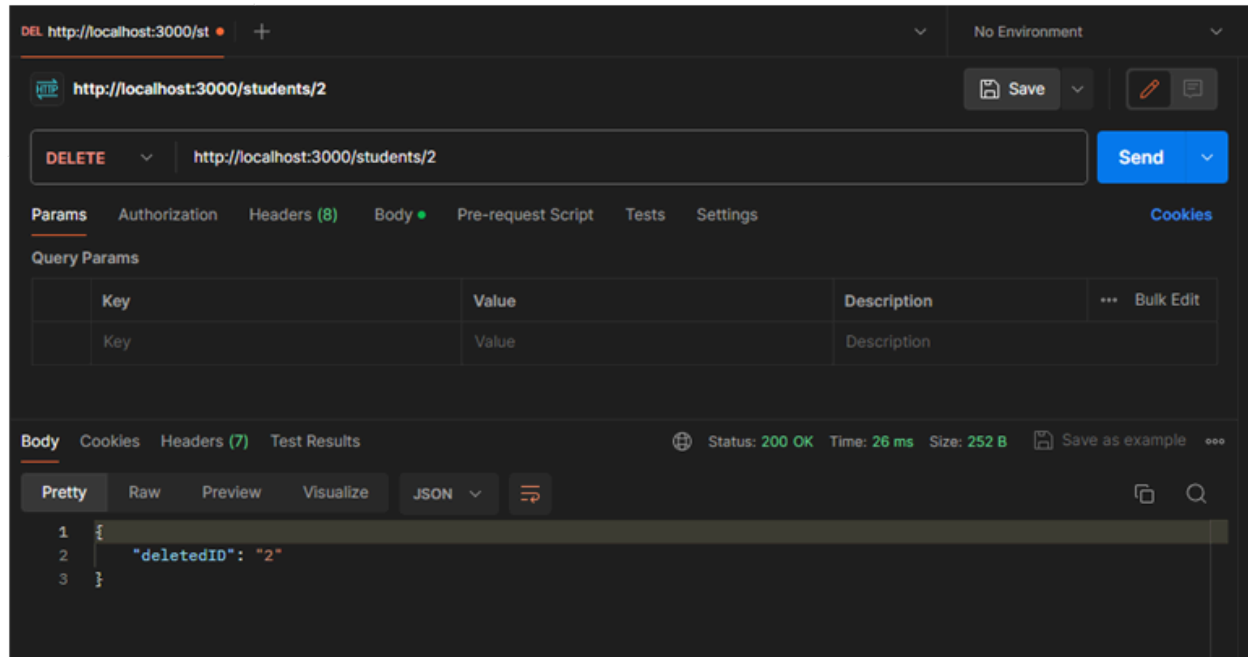
GET: #all Students

- Set the request type to GET.
- Enter the URL: `http://localhost:3000/students`.
- Click on the **"Send"** button
- You should receive a response with details of all students in your SQLite database.



DELETE:

- Set the request type to DELETE.
- Enter the URL for the student you want to delete (replace: id with an actual student ID): *http://localhost:3000/students/:id*
- Place instead of ID which replace with number that is ID to be deleted.
- Then click **send**



PUT:

- Set the request type to PUT.
- Enter the URL for the student you want to delete (replace: id with an actual student ID): *http://localhost:3000/students/:id*
- Go to the **"Body"** tab.
- Select raw and set the body to JSON format

PUT http://localhost:3000/st

No Environment

http://localhost:3000/students/1

Save

PUT

http://localhost:3000/students/1

Send

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Beautify

```
1 {
2   "name": "Madhu",
3   "age": 22,
4   "grade": "A+"
5 }
```

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 43 ms

Size: 252 B

Save as example

Pretty

Raw

Preview

Visualize

JSON

```
1 {
2   "updatedID": "1"
3 }
```